



INTELLIGENCE RESOURCE MANAGER · MCP SERVER · ROCKETRIDE

Leverage

One frontier brain. *An elastic workforce.*

\$0.00 PAID INFERENCE

88 TESTS GREEN

Verified OR NOTHING

The intelligence resource manager under the model you already pay for. Cheaper models do the work. The repository's own tests decide what counts.

LIVE USELEVERAGE.VERCEL.APP TESTS 97 GREEN CI PASSING MCP 5 TOOLS
ROCKETRIDE STAGING VERIFIED LICENSE MIT

Paid inference, five recorded missions	\$0.00
Same workload at frontier rates, estimated	\$0.24
Chat product from a test spec	28 / 28 tests
Billing SaaS, five modules, one mission	23 / 23 tests
Plan written by a free model	45.7 s
Workers replaced from a checkpoint	2 of 2 resumed



Built for the RocketRide x SCU Buildathon
Cloud workers run as RocketRide pipelines. Credits read from billing.

It is 2 a.m. The suite is red.

You paste the failing test into your best model. Third time tonight. It fixes one thing and breaks another. Then the rate limit lands, and you sit there waiting for a quota to reset.

Everyone who has shipped with an AI in the loop knows this night.

What if the smart one delegated?

Leverage sits under the model you already pay for. That model keeps the plan. A team of cheaper models takes the grind: the boilerplate, the retries, the test that has to go green.

Decides

your frontier model

Does

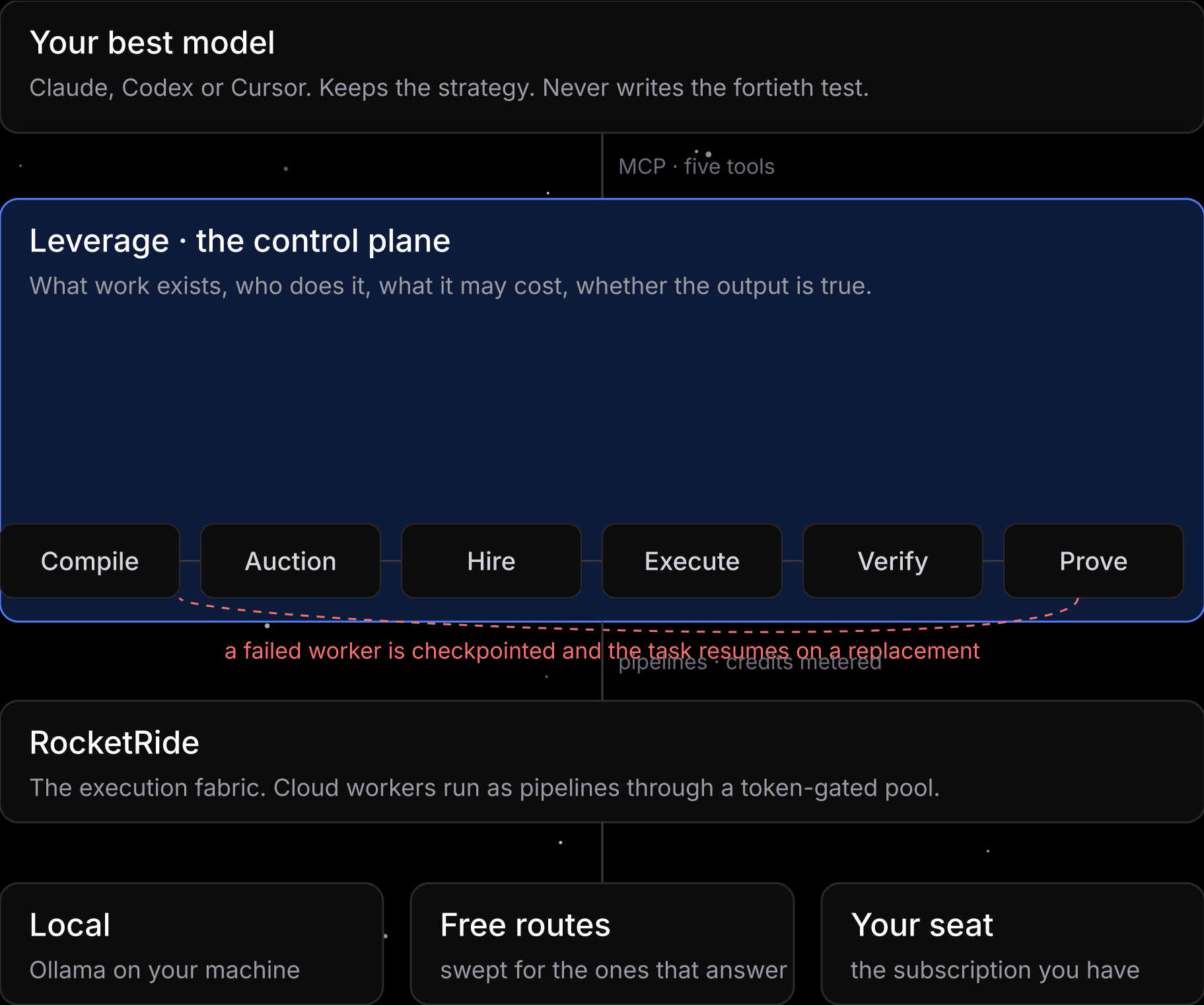
local, free and subscription models

Judges

the repository's own tests

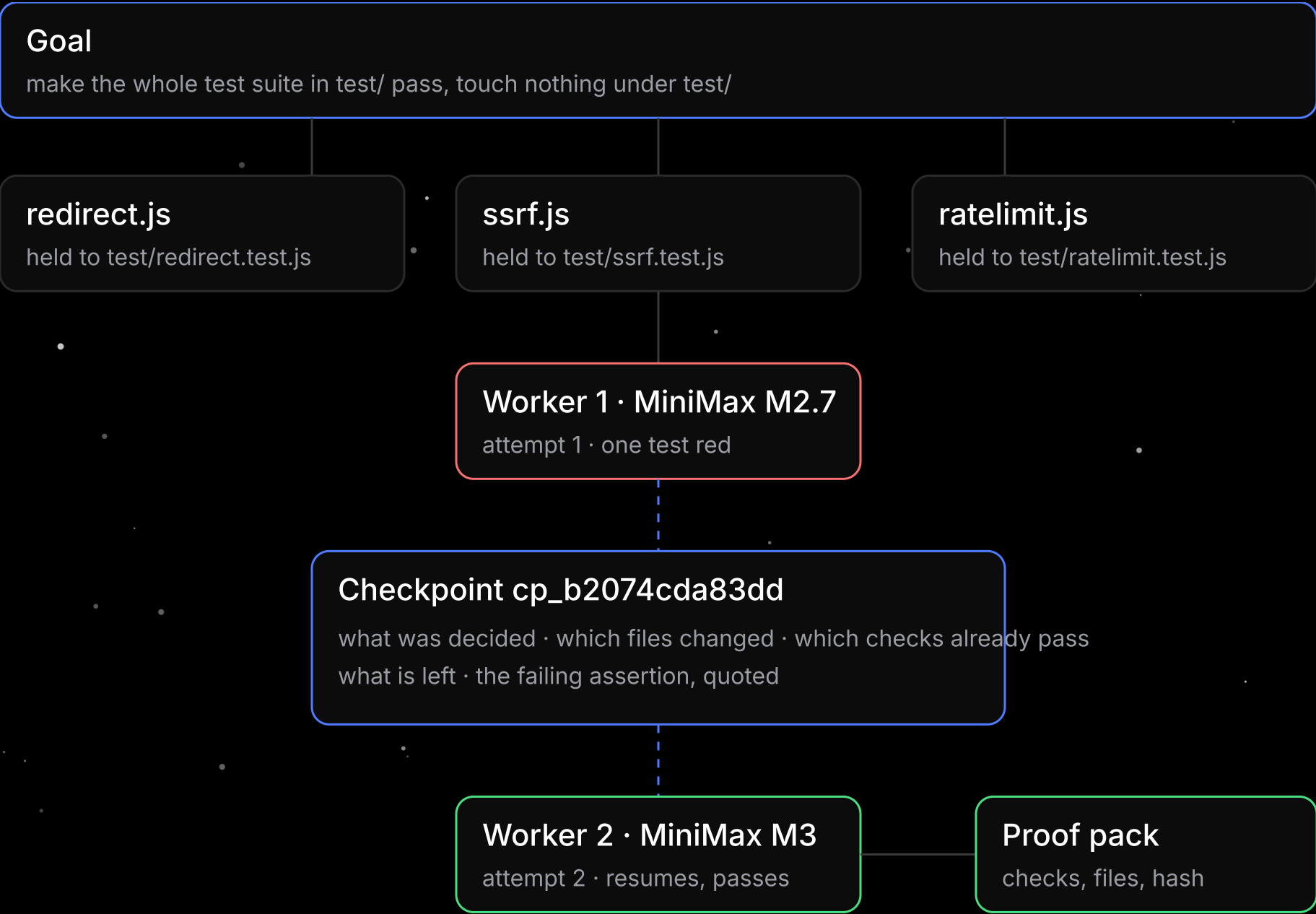
The harness.

Six stages between a goal and a proof, and one loop that runs when something breaks.



A worker dies. The work does not.

When a model hits a limit or fails a test, Leverage writes down what it understood and hands that note to the next one. Nobody starts from zero.



One of the two handoffs in mission LVR-e6443739. The second worker never re-read the whole repository.

The tests are the boss.

A task is finished when the test runner says so, not when a model says so. After the last task, the whole suite runs once more. Everything that passed is written into a proof pack you can read.

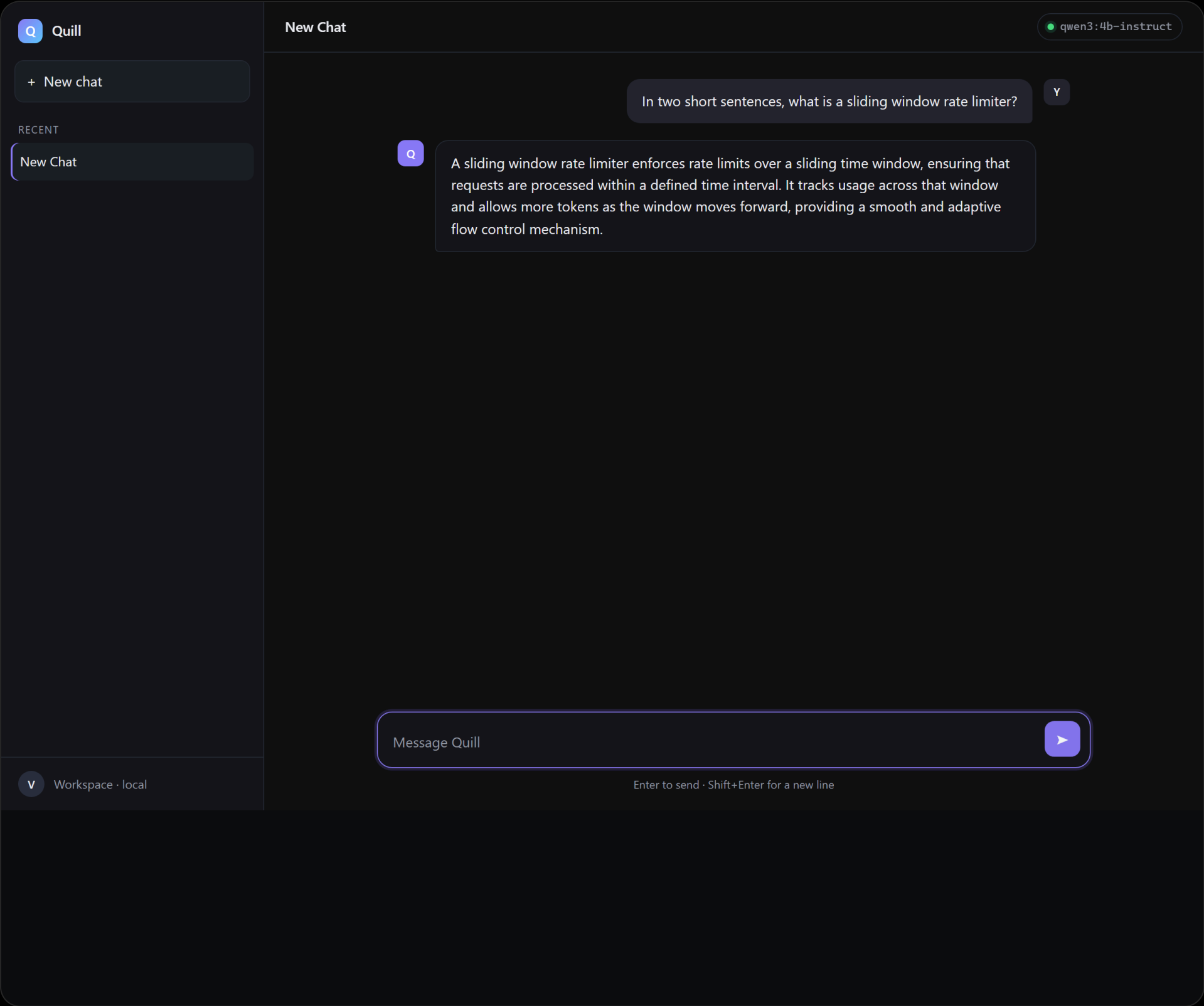
Exit 0	or it is not done
--------	-------------------

Whole suite	runs again at the end
-------------	-----------------------

Proof pack	checks, files, hash, spend
------------	----------------------------

It built a chat product.

We wrote the tests for a Claude-style assistant: conversations, a context window, rate limits, a model adapter, an HTTP server, then settings, projects, attachments and streaming. Workers wrote every module; the surface was given as a shell, the way a canvas is. This is Quill, answering from a model on the laptop.



Then a billing SaaS, five agents, one mission.

Plans and entitlements, usage metering, prorated invoices, signed webhooks, tenant isolation. Five independent modules, so the planner cut five tasks and hired a worker for each, two at a time. This is the console when they were done.

Leverage

Overview

New mission

Live run

Missions

Models

Providers

MISSION LVR-C65A1C37

COMPLETED

Build Meterly, the billing core of a usage-based SaaS, so that the whole test suite in test/ passes: src/plans.js, src/meter.js, src/invoice.js, src/webhooks.js and src/tenants.js, each described in README.md. The five modules are independent, so plan them as five separate tasks with no dependencies between them. Node built-ins only, CommonJS. Do not modify any file under test/.

plan written by Nemotron 3 Super 120B via NVIDIA (free) · 5 tasks in 19.1s

event stream complete · budget \$0.00 hard · privacy prefer-local

QUALITY	PAID SPEND	ELAPSED	TASKS	ACTIVE WORKERS	FRONTIER-EQUIV (estimated)
100	\$0.00	41.6s	5/5	0	\$0.0472

TASK GRAPH

BACKEND
Implement src/plans.js
PASSED

BACKEND
Implement src/meter.js
PASSED

BACKEND
Implement src/invoice.js
PASSED

BACKEND
Implement src/webhooks.js
PASSED

BACKEND
Implement src/tenants.js
PASSED

PLAN

written by a model

planner Nemotron 3 Super 120B via NVIDIA · free

took 19.1s · 2,776 in, 1,241 out

Implement src/plans.js

held to: All plans tests pass

Implement src/meter.js

held to: All meter tests pass

Implement src/invoice.js

held to: All invoice tests pass

Implement src/webhooks.js

held to: All webhooks tests pass

WORKFORCE

Backend Engineer
MiniMax M3 via OpenRouter free route
100% task fit · 25% verified success over 73 prior jobs · free route, no spend
context 479 tok \$0.0000

Backend Engineer
MiniMax M3 via OpenRouter free route
100% task fit · 24% verified success over 72 prior jobs · free route, no spend
context 468 tok \$0.0000

Backend Engineer
MiniMax M3 via OpenRouter free route
100% task fit · 23% verified success over 71 prior jobs · free route, no spend
context 695 tok \$0.0000

Backend Engineer
MiniMax M3 via OpenRouter free route
100% task fit · 22% verified success over 70 prior jobs · free route, no spend

Then, on the live site, one button.

A repository with no plan at all. A model writes the task graph, three workers are hired, three tasks pass their own tests, the whole suite runs green, and the RocketRide credits are read from billing before and after: 4129.1 to 4117.1.

Leverage

Overview

New mission

Live run

Missions

Models

Providers

LIVE

Run a mission

Everything else on this deployment is a recording of something that happened. This page is not. A real mission, real workers on RocketRide through the hosted pool, real verification, while you watch.

FINISHED105s3 hired3 passedcredits 4129.1 → 4117.1

LVR-c80e1b24

83.2sproof.check src/slug.js exists: PASS: src/slug.js exists

83.2sproof.check slugify tests pass: PASS: i duration_ms 90.59232

83.2sverification.passed "Implement slugify function" verified, quality 100

83.2stask.completed Task completed: Implement slugify function

101.3sworker.progress MiniMax M2.7 via OpenRouter wrote 1 file(s)

101.3sverification.started Verifying "Implement wordcount functions"

101.4sproof.check src/wordcount.js exists: PASS: src/wordcount.js exists

101.4sproof.check wordcount tests pass: PASS: i duration_ms 99.882291

101.4sverification.passed "Implement wordcount functions" verified, quality 100

101.4stask.completed Task completed: Implement wordcount functions

102.0sproof.check Whole test suite (npm test): PASS: i duration_ms 273.169075

102.0smission.completed Mission verified: 3/3 tasks passed, whole suite green

102.0sworker.progress RocketRide credits: 4129.1 before, 4117.1 after, 12 used

Reset

YOUR RUN, IN THE SAME VIEW EVERY RECORDED MISSION USES

Save this run as JSON

MISSION LVR-C80E1B24

Implement src/ so that the whole test suite in test/ passes: the tests import src/greet.js, src/slug.js and src/wordcount.js. Do not modify any file under test/. Budget: \$0. Quality: production.

committed plan · 3 tasks

COMPLETED

YOUR RUN · VIEW ONLY

event stream complete · budget \$0.00 hard · privacy cloud-allowed

QUALITY	PAID SPEND	ELAPSED	TASKS	ACTIVE WORKERS	FRONTIER-EQUIV (estimated)
100	\$0.00	101.4s	3/3	0	\$0.0125

Things people hand to it.

From the missions it has actually run.

The failing suite before standup

a four-task repo, seventeen tests, three worker failures survived

The physics for a small game

vector maths, a seeded spawner, a state machine, 22 tests green

A chat product from a test spec

six modules, one mission, running against a local model the same night

Three security guards from one chat message

open redirect, request forgery, rate limiting, planned in 45 seconds, every test green

It costs what you already pay.

Local models on your laptop. Free hosted routes. The subscription seat you have. Set the budget to zero and paid routes are not discouraged, they are removed from the room before the auction starts.

Local

Ollama, on your machine

Free

hosted routes, swept for the ones that answer

Yours

the seat you already pay for

Press the button.

A real mission runs on the live site while you watch. Planned by a model, verified by its tests.

useleverage.vercel.app